UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería
Diseño de Lenguajes de Programación
Semestre I – 2026

Proyecto 1

Integrantes:
Nájera Marakovits, Ingrid Nina Alessandra - 231088
Ramírez Velásquez, Diego Alejandro – 23601
Eliazar José Pablo Canastuj Matías - 23384

Enlace al repositorio: https://github.com/can23384/Proyecto01_Generador-de-analizadores-l-xicos

1. Descripción del proyecto

Este proyecto consiste en la implementación de un generador de analizadores léxicos, el cual recibe como entrada un archivo escrito en YALex y produce automáticamente un analizador léxico funcional.

El sistema desarrollado permite:

- Interpretar especificaciones léxicas.
- Construir autómatas finitos deterministas (AFD).
- Generar un lexer independiente.
- Detectar tokens y errores léxicos.

El sistema también genera diagramas de estados y un archivo ejecutable en JavaScript.

2. Objetivos

Objetivo general

- Implementar un generador de analizadores léxicos basado en especificaciones YALex.

Objetivos específicos

- Aplicar teoría de análisis léxico.
- Implementar construcción de AFD.
- Generar analizadores independientes.
- Detectar errores léxicos.

3. Diseño del software

El sistema está dividido en módulos que siguen una arquitectura por etapas:

3.1. Parser de YALex

- Archivo: yalex_parser.py
- Lee el archivo .yal Procesa: definiciones let reglas rule Maneja errores de sintaxis con mensajes detallados

3.2 Conversión de expresiones

- Archivo: yalex_converter.py

- Convierte expresiones YALex a formato interno Maneja: clases de caracteres rangos negaciones Normaliza símbolos especiales

3.3 Motor de expresiones regulares

- Archivo: regex_engine.py

Implementa:

- Tokenización de regex
- Conversión a postfix (Shunting Yard)
- Construcción de árbol sintáctico
- Cálculo de:
  - nullable
  - firstpos
  - lastpos
  - followpos
- Construcción directa de AFD
- Minimización del AFD
- No se usan librerías de regex, cumpliendo la restricción del proyecto.

3.4 Construcción del lexer

- Archivo: lexer_builder.py
- Genera los DFAs para cada regla
- Aplica:
  - regla de máximo lexema
  - prioridad por orden
- Maneja errores léxicos

3.5 Generación de código

- Archivo: codegen.py
- Genera automáticamente:
  - lexer_generado.js
- Convierte acciones YALex a JavaScript

Cumple con:

- El lexer generado es independiente del generador.

3.6 Ejecución principal

- Archivo: main.py
- Orquesta todo el proceso:
  1. Leer YALex

2. Construir lexer
3. Analizar archivo de entrada
4. Mostrar tokens y errores
5. Generar diagramas y código

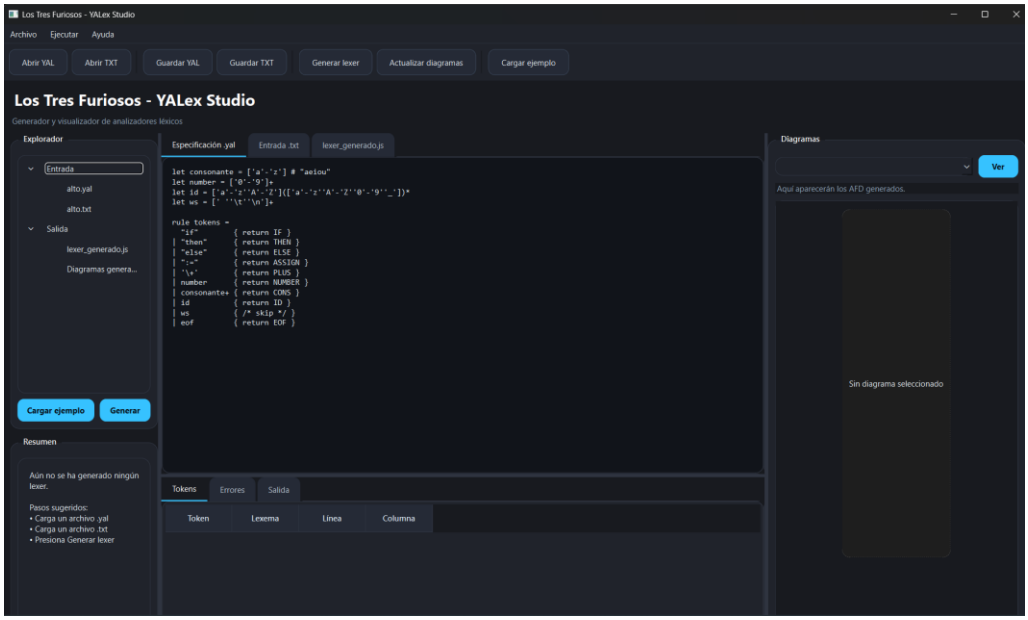
3.7 Manejo de errores

- Archivo: errors.py
- Maneja:
  - errores de especificación
  - errores léxicos
- Incluye mensajes detallados con línea y columna

3.8 Utilidades de tokens

- Archivo: token_utils.py
- Genera nombres de tokens automáticamente
- Interpreta acciones (return, print, etc.)

3.9 Interfaz gráfica del sistema



El sistema cuenta con una interfaz gráfica amigable que permite al usuario cargar archivos YALex, ejecutar el generador y visualizar los resultados de forma intuitiva.

4. Flujo de funcionamiento

1. Se lee el archivo `.yal`
2. Se parsea la especificación

3. Se convierten las expresiones
4. Se construyen AFDs
5. Se minimizan los autómatas
6. Se genera el lexer
7. Se analiza el archivo de entrada
8. Se imprimen tokens o errores

5. Entradas y salidas

Entrada del generador

- Archivo .yal

Salidas del generador

- Diagramas de estados (.dot / .png)
- Archivo lexer_generado.js

Entrada del lexer

- Archivo de texto plano

Salida del lexer

- Tokens reconocidos
- Errores léxicos

6. Pruebas realizadas

El proyecto incluye pruebas con tres niveles de complejidad:

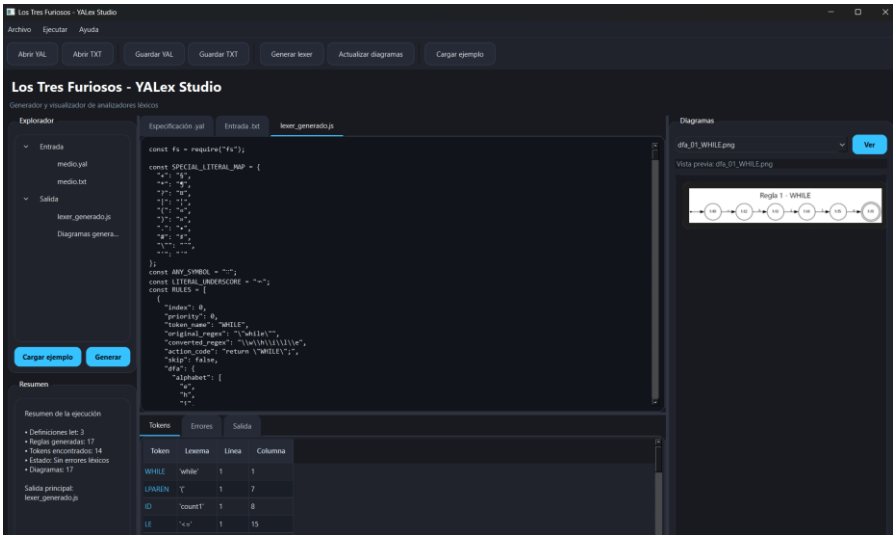
- Baja
- Media
- Alta

Esto cumple con lo requerido en la rúbrica.

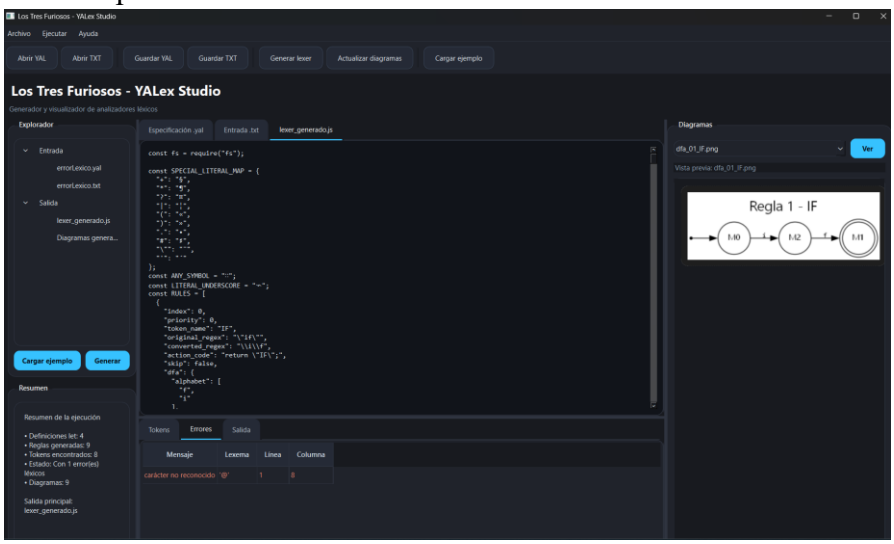
Cada prueba valida:

- reconocimiento de tokens
- detección de errores
- modificación del YALex

Ejemplo de uso con medio:



Y también se probó con errores



7. Observaciones

- El proyecto implementa completamente la teoría vista en clase.
- El sistema es modular y extensible.
- Se puede adaptar para nuevos lenguajes fácilmente.

8. Conclusiones

Se logró implementar un generador de analizadores léxicos funcional que:

- Aplica correctamente la teoría de autómatas.
- Genera analizadores independientes.
- Detecta tokens y errores de manera eficiente.

El proyecto cumple con los requisitos técnicos y académicos establecidos